



VAMOS App

Planes de Pruebas Funcionales

Grupo N° 5

Versión: 1

Fecha: 22/06/2026

HOJA DE CONTROL

Cliente	Usuarios Locales de Providencia.		
Proyecto	Vamos?!		
Entregable	Planes de Pruebas Funcionales		
Autor	Equipo VAMOS App		
Versión/Edición	1.0	Fecha Versión	21/06/2026
Aprobado por		Fecha Aprobación	21/06/2026

REGISTRO DE CAMBIOS

Versión doc	Causa del Cambio	Responsable del Cambio	Fecha del Cambio
001	Versión inicial	Freddy G. Galarza / Pablo Mery / Luis Núñez	21/06/2026

CONTROL DE DISTRIBUCIÓN

Nombre y Apellidos
Freddy G. Galarza
Pablo Mery
Luis Núñez

ÍNDICE

1 INTRODUCCIÓN	3
1.1 Objeto	3
1.2 Alcance	3
2 TRAZABILIDAD DE CASOS DE PRUEBAS – REQUISITOS	4
3 DEFINICIÓN DE LOS CASOS DE PRUEBAS	5
4 ESTRATEGIA DE EJECUCIÓN DE PRUEBAS	8
4.1 Entorno y Base de Datos de Pruebas	8
5 ANEXOS	9
5.1 Evidencias de ejecución	9
5.2 Extras	12
6 GLOSARIO	13

1 INTRODUCCIÓN

1.1 Objeto

El objetivo de este documento es recoger los casos de pruebas que verifican que el sistema satisface los requisitos especificados. Deberá contener la definición de los casos de prueba, la matriz de trazabilidad entre casos de pruebas y requisitos, y la estrategia a seguir en la ejecución de las pruebas.

1.2 Alcance

El alcance de este plan abarca la validación integral de los módulos funcionales y de integración que componen la aplicación "VAMOS!". Las pruebas contemplan la verificación del flujo de autenticación delegada mediante Google OAuth 2.0 y las restricciones lógicas de edad (mayores de 18 años). Asimismo, se audita el correcto funcionamiento de información dinámica orientado a la cartelera de Providencia, la persistencia de datos en PostgreSQL dentro del entorno AWS EC2, la capacidad de respuesta semántica del agente Vambot (IA), y la correcta ejecución del ruteo nativo con Google Maps. Para asegurar la fiabilidad en la gestión de grupos logísticos concurrentes, se generaron y utilizaron cuentas de prueba aisladas para evitar colisiones de sesión y garantizar la persistencia de estados.

2 TRAZABILIDAD DE CASOS DE PRUEBAS – REQUISITOS

	RF-01 (Autenticación y Usuarios)	RF-02 (Exploración)	RF-03 (Vambot)	RF-04 (Gestión Grupos)	RF-05 (Tracking Estados)	RF-06 (Ruteo)	RF-07 (Guardados / Confirmados)
<CP-1>	X						
<CP-2>		X					
<CP-3>			X				
<CP-4>				X			X
<CP-5>					X		
<CP-6>						X	

3 DEFINICIÓN DE LOS CASOS DE PRUEBAS

Autenticación OAuth y Restricción de Edad	<Código del CP>	CP-01
	¿Prueba de despliegue?	Si
Descripción: Validar que el sistema procesa el login nativo con Google y bloquea correctamente la creación de perfiles si el usuario indica una fecha de nacimiento que resulta en una edad inferior a 18 años.		
Prerrequisitos Aplicación móvil compilada y API REST recibiendo peticiones en AWS.		
Pasos: En la pantalla obligatoria de edad, probar el <i>Flujo Exitoso</i> : Ingresar una fecha de nacimiento que resulte en más de 18 años (ej. 1998) y confirmar. 3. Repetir el proceso para probar el <i>Flujo de Restricción</i> : Ingresar una fecha de nacimiento que resulte en < 18 años (ej. 2012) y confirmar.		
Resultado esperado: Para el flujo exitoso, el sistema crea el registro en PostgreSQL, genera el token de sesión y redirige al mapa principal. Para el flujo de restricción, el sistema bloquea el flujo, no guarda datos en la BD y muestra la alerta "Lo sentimos, debes tener al menos 18 años".		
Resultado obtenido: El sistema diferencia correctamente los rangos de edad, permitiendo el ingreso a usuarios válidos y rechazando a los menores cumpliendo la regla de negocio. (Pass).		

Caso 1

Obtención Dinámica y Renderizado de Eventos	<Código del CP>	CP-02
	¿Prueba de despliegue?	Si
Descripción: Verificar que se procese dinámicamente los eventos del mes actual desde la web de Providencia y los renderice como marcadores en el mapa nativo.		
Prerrequisitos Instancia EC2 funcionando y CronJob habilitado en el servidor.		
Pasos: Ejecutar el script para obtener la información de los eventos de la municipalidad de Providencia. Ejecutar el orquestador de ingesta de datos (Data Seeder). Iniciar sesión en la app VAMOS!. Visualizar la pestaña "Mapa".		
Resultado esperado: El sistema extrae los datos sin errores de sintaxis, realiza el Upsert en PostgreSQL y la aplicación renderiza los marcadores geolocalizados del mes en curso sin necesidad de intervención manual.		

Resultado obtenido:

Marcadores del mes actual (junio) procesados y renderizados correctamente. (Pass).

Caso 2

Asistencia Inteligente (Vambot)	<Código del CP>	CP-03
	¿Prueba de despliegue?	No
Descripción: Comprobar que el microservicio FastAPI procesa prompts de lenguaje natural y devuelve recomendaciones utilizando exclusivamente el contexto de eventos extraídos.		
Prerrequisitos API Key de Gemini configurada en el .env del servidor.		
Pasos: Abrir el BottomSheet de Vambot. Escribir: "Busco una actividad familiar para este fin de semana". Enviar consulta.		
Resultado esperado: Vambot responde en menos de 10 segundos con una recomendación semántica basada en la tabla de eventos de Providencia.		
Resultado obtenido: Respuesta coherente y formateada obtenida dentro del límite de tiempo. (Pass).		

Caso 3

Gestión Logística y Compartición de Códigos	<Código del CP>	CP-04
	¿Prueba de despliegue?	Si
Descripción: Validar el flujo de confirmación de un evento, creación de un grupo privado e invocación del Intent nativo para compartir el código mediante aplicaciones externas (WhatsApp, Instagram).		
Prerrequisitos Usuario autenticado con JWT válido.		
Pasos: Marcar un evento como "Confirmado". Ir a Mis Eventos -> Unirse a un grupo -> Crear. Presionar "Invitar" para capturar el invite_code.		
Resultado esperado: El backend genera un código único y la aplicación levanta el menú nativo de compartición del sistema operativo (Share.share) permitiendo enviar el texto a WhatsApp/IG.		
Resultado obtenido: Código generado y enviado exitosamente a través de aplicaciones de terceros. (Pass)		

Caso 4

Sincronización Concurrente de Estados	<Código del CP>	CP-05
	¿Prueba de despliegue?	No
Descripción: Confirmar que la actualización de estado ('en camino', 'llegué') de un participante del grupo impacte en la base de datos y sea visible para el resto de los integrantes.		
Prerrequisitos Dos cuentas de prueba aisladas (Usuario A y Usuario B) pertenecientes al mismo grupo logístico.		
Pasos: Usuario A cambia su estado a "En camino" desde el modal inferior. Usuario B refresca la vista del grupo.		
Resultado esperado: El PATCH en la API actualiza el estado transaccional en AWS y el Usuario B percibe el cambio visual del ícono de estado del Usuario A.		
Resultado obtenido: Persistencia de estado validada concurrentemente sin cruce de sesiones. (Pass).		

Caso 5

Ruteo Nativo (Google Maps)	<Código del CP>	CP-06
	¿Prueba de despliegue?	Si
Descripción: Validar que al solicitar instrucciones de llegada, el sistema procese el origen y el destino para trazar la ruta geométrica visual en la interfaz.		
Prerrequisitos GPS del dispositivo encendido. Permisos de ubicación otorgados.		
Pasos: Seleccionar un marcador de evento en el mapa. Presionar el botón "Cómo llegar".		
Resultado esperado: La aplicación consume la Directions API de Google Maps y dibuja una ruta exacta desde la ubicación del usuario hasta las coordenadas del evento.		
Resultado obtenido: Ruta calculada y trazada correctamente sobre el mapa base. (Pass).		

Caso 6

4 ESTRATEGIA DE EJECUCIÓN DE PRUEBAS

4.1 Entorno y Base de Datos de Pruebas

Para la ejecución de los casos de prueba descritos, se empleó un entorno de base de datos relacional alojado directamente en la infraestructura en la nube (AWS EC2). El motor utilizado es PostgreSQL operando bajo contenedores de Docker, el cual incorpora la extensión pgvector para habilitar el procesamiento de similitud semántica requerido por el módulo de IA (Vambot).

El entorno de datos se estructuró bajo los siguientes parámetros:

Poblado de Datos (Data Seeding): La información transaccional de los eventos no fue insertada manualmente. Se utilizó la base de datos resultante de la ingesta automatizada del script en Python, el cual extrajo, procesó e insertó mediante lógica Upsert la cartelera real y vigente de la Municipalidad de Providencia.

Usuarios de Prueba: Para auditar los módulos logísticos de interacción grupal y el tracking de estados (RF-04 y RF-05), se generaron cuentas de prueba aisladas (ej. test_user_A, test_user_B). Esto garantizó un entorno aséptico para validar la concurrencia de transacciones en la base de datos sin colisión de sesiones ni sobrescritura de estados.

Integridad y Resiliencia: Se configuró un entorno paritario local para el respaldo de contingencia (backup_produccion.sql) y se asignó memoria SWAP a la instancia en la nube para garantizar que el motor de la base de datos no sufriera caídas (Out Of Memory) durante las pruebas de estrés.

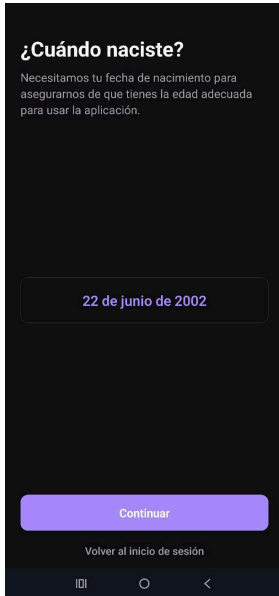
La estrategia se basó en un enfoque incremental alineado a los Sprints del proyecto, asegurando paridad entre el entorno local y el despliegue en contenedores Docker en AWS.

	Ciclo 1 (Unitarias API)	Ciclo 2 (Integración App/Mapa)	Ciclo 3 (Regresión, Scraper y Concurrencia)
<CP-1>	X	X	X
<CP-2>	X		X
<CP-3>		X	X
<CP-4>		X	X
<CP-5>			X
<CP-6>		X	X

5 ANEXOS

5.1 Evidencias de ejecución

Caso 1



¿Cuándo naciste?

Necesitamos tu fecha de nacimiento para asegurarnos de que tienes la edad adecuada para usar la aplicación.

22 de junio de 2002

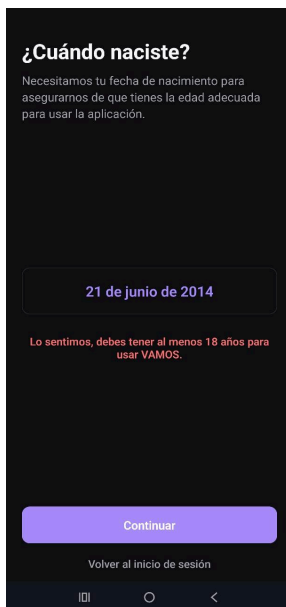
Continuar

Volver al inicio de sesión

Paso-1 Prueba Flujo exitoso

pablomery2010@gmail...	2002-06-22
------------------------	------------

Paso-2 confirmación Dentro de BD



¿Cuándo naciste?

Necesitamos tu fecha de nacimiento para asegurarnos de que tienes la edad adecuada para usar la aplicación.

21 de junio de 2014

Lo sentimos, debes tener al menos 18 años para usar VAMOS.

Continuar

Volver al inicio de sesión

Paso 3 Pruebas de flujo erroneo

Caso 2

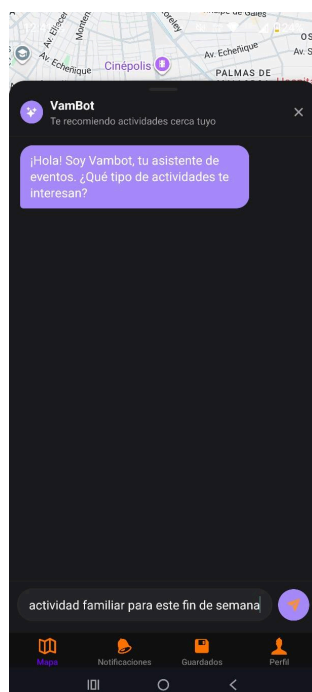
```
(venv_scripts) [ec2-user@ip-10-0-3-31 Back-End]$ python DataSeeder/sincronizador_bd.py
=== INICIANDO WORKER DE SINCRONIZACIÓN (POSTGRESQL) ===
-> Se cargaron 71 eventos desde el Staging Area.
-> Sincronizando con PostgreSQL...
✅ ¡Éxito! Base de datos PostgreSQL actualizada con 71 registros.
```

Correcto llenado de BD

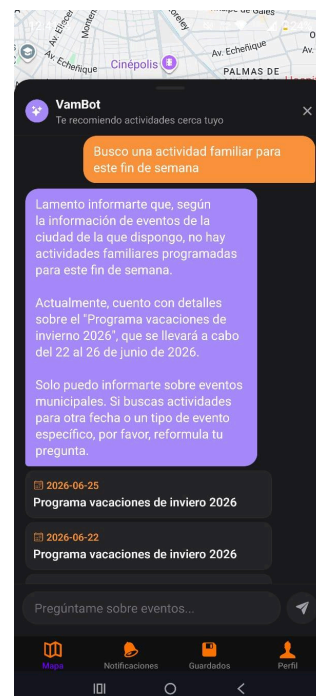
nombre_evento character varying (255)	fecha date
Actividades deportivas	2026-06-22
Actividades deportivas	2026-06-23
Actividades deportivas	2026-06-24
Actividades deportivas	2026-06-25
Actividades deportivas	2026-06-26
Actividades deportivas	2026-06-27
Actividades deportivas	2026-06-28
Actividades deportivas	2026-06-29
Actividades deportivas	2026-06-30
Feria Mercado Verde Urbano	2026-06-22

Confirmación de datos guardados en la BD

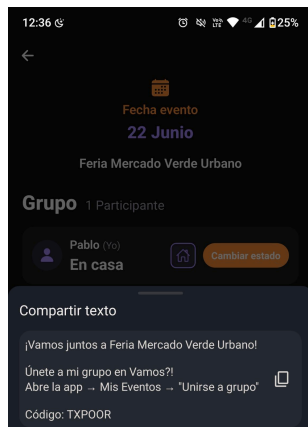
Caso 3



Prueba Consulta

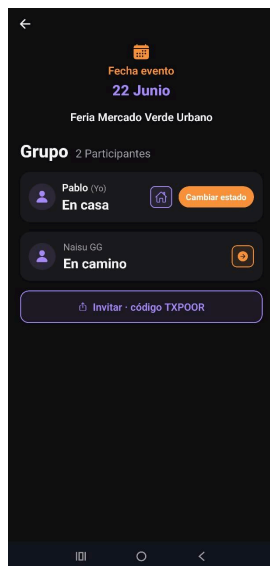


Respuesta de VamBot

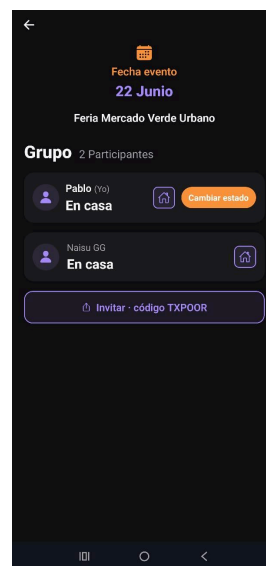


Creación de invitación

Caso 5

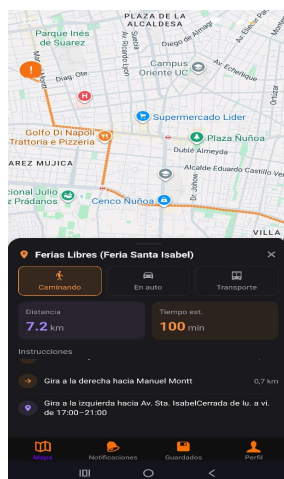


Vista persona A



Vista Persona B

Caso 6



Correcto Funcionamiento de Ruteo Nativo

5.2 Extras

Durante los ciclos de pruebas se detectaron comportamientos sistémicos que derivaron en refactorizaciones clave para garantizar los más altos estándares de calidad de la industria (Disponibilidad y Tolerancia a Fallos):

Mitigación de OOM (Out Of Memory) en AWS: Tras pruebas de estrés iniciales, se detectó lentitud y caídas abruptas en la capa gratuita de EC2 debido al consumo de RAM de los contenedores simultáneos. Mejora: Se configuró un archivo de memoria virtual temporal (SWAP) directamente en la capa del sistema operativo Linux para expandir la capacidad y mantener la API siempre disponible, respaldando esto con copias de imágenes en Amazon ECR.

Refactorización a Scraper Dinámico (Compleitud): En los primeros ciclos (abril/mayo) el script semilla requería ajustes manuales de URL. Mejora: Se refactorizó el código Python para que la extracción se adapte dinámicamente al mes en curso, previniendo caídas silenciosas en el abastecimiento de datos del mapa y del LLM.

Ambientes Aislados (Concurrencia): Para certificar la trazabilidad y separación de datos, las pruebas del Módulo Logístico se ejecutaron creando cuentas de prueba aisladas. Esto permitió simular interacciones reales entre amigos (cruces de peticiones HTTP) confirmando que no existen fugas de información ni sobrescritura de estados de sesión.

Sugerencias de Usuarios (Feedback): Durante las sesiones de prueba y observación, los usuarios manifestaron el deseo de poder subir fotografías de los eventos a los que asisten, además de contar con un chat integrado dentro de los grupos logísticos. Si bien se aclaró que estas características no se implementarán por el momento, el equipo documentó este feedback para tenerlo en cuenta como posibles requerimientos en futuras versiones del proyecto.

6 GLOSARIO

Término	Descripción
OAuth 2.0	Protocolo estándar utilizado para la delegación segura de autenticación (Google Sign-In).
SWAP	Espacio en el disco duro utilizado como memoria RAM virtual para prevenir caídas de servidor por sobrecarga.
ECR (Elastic Container Registry)	Servicio de Amazon para almacenar y recuperar imágenes de contenedores Docker de forma segura.
JWT (JSON Web Token)	Estándar para transmitir información de sesión entre cliente y servidor de forma segura, sin almacenar estado en el servidor.
IdToken	Token de identidad retornado por Google tras autenticar al usuario; contiene correo, nombre y foto de perfil.
API REST	Interfaz de comunicación entre cliente y servidor mediante peticiones HTTP estándar (GET, POST, PATCH, DELETE) con respuestas en JSON.
FastAPI	Framework web Python de alto rendimiento utilizado para el microservicio de Vambot, por su soporte asíncrono y velocidad.
BottomSheet	Componente de UI móvil que emerge desde la parte inferior de la pantalla para mostrar detalles sin perder el contexto visual.
CronJob	Tarea programada que se ejecuta automáticamente en intervalos definidos
Data Seeder	Proceso que transforma los datos y los inserta o actualiza en la base de datos mediante lógica Upsert.
Upsert	Operación de base de datos que inserta un registro si no existe, o lo actualiza si ya existe, evitando duplicados.
Embedding	Representación matemática vectorial de un texto que captura su significado semántico, permitiendo comparar similitud entre frases.
LLM (Large Language Model)	Modelo de lenguaje de gran escala entrenado para procesar y generar texto natural; en este proyecto se usa Google Gemini.
Vambot	Asistente virtual conversacional de VAMOS App, basado en IA, que recomienda eventos a partir de las consultas del usuario.
Prompt	Texto ingresado por el usuario al agente Vambot para solicitar una recomendación de eventos.
AWS EC2	Servicio de Amazon Web Services que provee instancias de servidores virtuales en la nube; usado para alojar el backend.
Docker / Docker Compose	Tecnología de contenerización que empaqueta aplicaciones con sus dependencias; Docker Compose orquesta múltiples contenedores como un sistema.
invite_code	Código alfanumérico único generado por el backend al crear un grupo logístico; permite a otros usuarios unirse al grupo.
Share.share (expo-sharing)	Método nativo de Expo que invoca el menú de compartición del sistema operativo Android para enviar texto a apps externas.
Free Tier	Capa gratuita de servicios en la nube (AWS, Google) con límites de uso mensual sin costo.